

Introdução à Programação PHP

Da sintaxe básica à criação de uma API REST com PHP puro — fundamentos essenciais para a disciplina.

PHP 8.x

Versão Atual

Back-end

Foco da Aula

4h

Duração

Professor: **Jackson Meires**

O que é PHP?

PHP (*PHP: Hypertext Preprocessor*) é uma linguagem de programação **server-side**, de código aberto, amplamente utilizada para desenvolvimento web. O código PHP é interpretado no servidor e o resultado é enviado ao cliente (normalmente HTML ou JSON).



Criado em 1994

Por Rasmus Lerdorf. Hoje gerenciado pelo grupo PHP.



Onipresente

Presente em ~77% dos servidores web com linguagem conhecida (W3Techs).



Rápido no aprendizado

Sintaxe familiar para quem conhece C, Java ou JavaScript.



Integração nativa

Suporte nativo a MySQL, PostgreSQL, SQLite, Redis e muito mais.

No nosso contexto: usaremos PHP para criar APIs REST que serão consumidas por um app React Native.

Ambiente de Desenvolvimento

Laragon (Windows) — stack tudo-em-um:

- **Apache** — Servidor HTTP
- **PHP 8.x** — Interpretador
- **MySQL** — Banco de dados
- **phpMyAdmin** — Interface visual BD

✓ Já instalado

laragon.org

Pasta dos projetos:

C:\laragon\www\

Acesse via browser:

http://localhost/pasta_projeto/

Fluxo de execução:

1. Browser faz requisição HTTP

2. Apache recebe e encaminha ao PHP

3. PHP executa o script (.php)

4. Resposta (HTML/JSON) é enviada ao browser

Primeiro Script PHP

Todo código PHP fica dentro das tags `<?php` e `?>`. Em arquivos puramente PHP (sem HTML), a tag de fechamento é opcional e até recomendada omitir.

hello.php

```
// Arquivo: C:\laragon\www\meu-projeto\hello.php
<?php

// Exibir texto no browser
echo "Olá, Mundo!";

// Exibir HTML
echo "<h1>Meu primeiro script</h1>";

// Função de debug: var_dump
var_dump(42);
var_dump("texto");
var_dump(true);

// print_r para arrays
print_r([1, 2, 3]);
```

⚠ **Atenção:** PHP é **sensível a maiúsculas** em nomes de variáveis (`$nome` ≠ `$Nome`), mas **não** em nomes de funções e palavras-chave.

Variáveis e Tipos de Dados

variaveis.php

```
<?php
// Variáveis começam com $
$nome  = "Maria";    // string
$idade = 25;         // int
$altura = 1.68;      // float
$ativo  = true;      // bool
$nulo   = null;      // null

// Verificar tipo
gettype($nome);      // "string"
is_int($idade);      // true
is_string($nome);    // true

// Casting (conversão)
$num  = (int) "42abc"; // 42
$str  = (string) 100;  // "100"
$bool = (bool) 0;     // false
```

Tipagem dinâmica — PHP infere o tipo automaticamente.

Tipo	Exemplo
string	"texto", 'texto'
int	42, -10
float	3.14, 1.0e3
bool	true, false
array	[1, 2, 3]
null	null

PHP 8 Declaration: você pode declarar tipos para tornar o código mais seguro: `function soma(int $a, int $b): int`

Strings e Interpolação

strings.php

```
<?php
$nome = "João";
$idade = 20;

// Aspas duplas: interpola variáveis
echo "Olá, $nome! Você tem $idade anos.";

// Aspas simples: literal (mais rápido)
echo 'Olá, $nome!'; // imprime $nome

// Concatenação com ponto (.)
echo "Nome: " . $nome . ", Idade: " . $idade;

// Funções úteis de string
strlen($nome);           // 4
strtoupper($nome);       // "JOÃO"
strtolower($nome);       // "joão"
str_replace("o","0",$nome); // "Jã0"
trim(" texto ");        // "texto"
substr($nome, 0, 2);     // "Jo"
str_contains($nome,"ão"); // true (PHP8)
```

Heredoc — strings multilinha:

```
$html = <<<EOT
<div>
    <h1>{$nome}</h1>
    <p>Idade: {$idade}</p>
</div>
EOT;
```

sprintf — formatação:

```
$msg = sprintf(
    "Nome: %s, Nota: %.1f",
    $nome,
    9.75
);
// "Nome: João, Nota: 9.8"
```

Estruturas de Controle

controle.php — if / switch

```
<?php
$nota = 7.5;

// if / elseif / else
if ($nota ≥ 9) {
    echo "Excelente!";
} elseif ($nota ≥ 6) {
    echo "Aprovado";
} else {
    echo "Reprovado";
}

// Operador ternário
$status = $nota ≥ 6 ? "Aprovado" : "Reprovado";

// Null coalescing (PHP 7+)
$nome = $_GET['nome'] ?? 'Anônimo';

// switch
switch ($nota) {
    case 10: echo "Perfeito"; break;
    case 9: echo "Ótimo"; break;
    default: echo "OK";
}
```

controle.php — loops

```
// for clássico
for ($i = 0; $i < 5; $i++) {
    echo $i . " ";
}
// 0 1 2 3 4

// while
$n = 1;
while ($n ≤ 5) {
    echo $n++ . " ";
}

// foreach (ideal para arrays)
$frutas = ["Maçã", "Banana", "Uva"];
foreach ($frutas as $fruta) {
    echo $fruta . "<br>";
}

// foreach com chave ⇒ valor
$ pessoa = ['nome' ⇒ 'Ana', 'idade' ⇒ 22];
foreach ($pessoa as $chave ⇒ $valor) {
    echo "$chave: $valor<br>";
}
```

Funções

funcoes.php

```
<?php
// Declaração básica
function somar($a, $b) {
    return $a + $b;
}
echo somar(3, 4); // 7

// Parâmetros com valor padrão
function cumprimentar($nome, $saudacao = "Olá") {
    return "$saudacao, $nome!";
}
echo cumprimentar("Pedro");           // Olá, Pedro!
echo cumprimentar("Pedro", "Oi");     // Oi, Pedro!

// Tipos declarados (PHP 7+)
function dividir(float $a, float $b): float {
    if ($b === 0.0) throw new Exception("Divisão por zero!");
    return $a / $b;
}
```

funcoes.php — arrow functions

```
// Funções anônimas (closure)
$dobrar = function($n) {
    return $n * 2;
};
echo $dobrar(5); // 10

// Arrow function (PHP 7.4+)
$triplicar = fn($n) => $n * 3;

// Funções de alta ordem (array)
$num = [1, 2, 3, 4, 5];

$pares = array_filter($num, fn($n) => $n % 2 === 0);
// [2, 4]

$dobrados = array_map(fn($n) => $n * 2, $num);
// [2, 4, 6, 8, 10]

$soma = array_reduce($num, fn($carry, $item) => $carry + $item, 0);
// 15
```


Arrays

arrays.php

```
<?php
// Array indexado
$notas = [8.5, 7.0, 9.5, 6.0];
echo $notas[0]; // 8.5

// Array associativo (como objeto JSON)
$aluno = [
    'nome'    => 'Ana',
    'matricula' => '2024001',
    'nota'    => 9.5,
];
echo $aluno['nome']; // Ana

// Array multidimensional
$turma = [
    ['nome' => 'Ana', 'nota' => 9.5],
    ['nome' => 'Bob', 'nota' => 7.0],
];
echo $turma[1]['nome']; // Bob

// Operações comuns
$notas[] = 10;           // adicionar
array_push($notas, 8);   // adicionar
array_pop($notas);       // remover último
sort($notas);            // ordenar
count($notas);           // quantidade
in_array(9.5, $notas);    // verificar existência
```

JSON ↔ Array — fundamental para APIs:

```
// Array para JSON
$dados = ['id' => 1, 'nome' => 'Ana'];
$json = json_encode($dados);
// '{"id":1,"nome":"Ana"}'

// JSON para Array associativo
$arr = json_decode($json, true);
echo $arr['nome']; // Ana

// JSON para Objeto stdClass
$obj = json_decode($json);
echo $obj->nome; // Ana
```

Dica: Ao criar APIs REST, sempre use `json_encode()` para responder e defina o header `Content-Type: application/json`.

Formulários e Superglobais

form.html

```
<form method="POST" action="processar.php">
  <input type="text" name="nome" placeholder="Nome">
  <input type="email" name="email" placeholder="Email">
  <button type="submit">Enviar</button>
</form>
```

processar.php

```
<?php
// $_POST recebe dados do formulário (method="POST")
$nome = $_POST['nome'] ?? '';
$email = $_POST['email'] ?? '';

// Sanitização (NUNCA confie no input do usuário!)
$nome = htmlspecialchars(trim($nome));
$email = filter_var($email, FILTER_SANITIZE_EMAIL);

// Validação
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Email inválido!";
    exit;
}

echo "Bem-vindo, $nome!";
```

Superglobal	Uso
\$_GET	Parâmetros da URL: ?nome=Ana
\$_POST	Dados enviados via formulário POST
\$_SERVER	Informações do servidor e da requisição HTTP
\$_SESSION	Dados persistentes entre requisições
\$_COOKIE	Cookies do navegador

Orientação a Objetos (POO)

Servico.php — Classe

```
<?php
class Servico {
    // Propriedades (atributos)
    private int $id;
    private string $titulo;
    private float $preco;

    // Construtor
    public function __construct(
        string $titulo,
        float $preco
    ) {
        $this->titulo = trim($titulo);
        $this->preco = $preco;
    }

    // Getters
    public function getTitulo(): string {
        return $this->titulo;
    }
    public function getPreco(): float {
        return $this->preco;
    }

    // Método
    public function toArray(): array {
        return [
            'titulo' => $this->titulo,
            'preco' => $this->preco,
        ];
    }
}
```

Usando a classe:

```
// Instanciar objeto
$s = new Servico("Hidráulica", 150.0);

echo $s->getTitulo(); // Hidráulica
echo $s->getPreco();  // 150

$arr = $s->toArray();
echo json_encode($arr);
```

Conceitos-chave:

- **Encapsulamento** — private, protected, public
- **Herança** — class Filho extends Pai
- **Interface** — contrato de métodos
- **Abstração** — classe abstrata

O Slim Framework usa intensamente POO — entender classes é fundamental!

Banco de Dados com PDO — Classe db

db.class.php — definição da classe

```
<?php
class db {
    private $host      = 'localhost';
    private $user       = 'root';
    private $password   = '';
    private $port       = '3306';
    private $dbname     = 'db_pweb_202x_x';
    private $table_name;
    private $conn; // conexão guardada para reutilizar

    public function __construct($table_name) {
        $this->table_name = $table_name;
        $this->conn = $this->connect(); // cria conexão uma única vez
    }

    // privado: apenas a própria classe pode chamar
    private function connect() {
        try {
            return new PDO(
                "mysql:host=$this->host;dbname=$this->dbname;port=$this->port;charset=utf8",
                $this->user, $this->password,
                [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
            );
        } catch (PDOException $e) {
            die('Erro na conexão: ' . $e->getMessage());
        }
    }

    public function all()           { /* SELECT * FROM tabela */ }
    public function find($id)       { /* SELECT ... WHERE id = ? */ }
    public function findBy($campo, $valor) { /* WHERE campo = ? */ }
    public function store($dados)   { /* INSERT dinâmico */ }
    public function update($id, $dados) { /* UPDATE ... WHERE id */ }
    public function destroy($id)    { /* DELETE WHERE id = ? */ }
    public function search($dados) { /* WHERE campo LIKE ? - $dados['tipo'] e $dados['valor'] */ }
}
```

uso — CRUD com a classe db

```
require_once 'db.class.php';

// Instanciar passando o nome da tabela
$db = new db('servicos');

// READ - listar todos
$servicos = $db->all();

// READ - buscar por id
$servico = $db->find(1);

// READ - buscar por campo específico
$s = $db->findBy('titulo', 'Elétrica');

// CREATE - array associativo (sem id)
$db->store([
    'titulo' => 'Elétrica',
    'preco'  => 200.00,
]);

// UPDATE - id separado dos dados
$db->update(1, [
    'preco' => 250.00,
]);

// DELETE - passar o id
$db->destroy(1);

// SEARCH - busca por campo e valor
$res = $db->search(['tipo' => 'titulo', 'valor' => 'hidráulica']);

// Mesma classe para qualquer tabela!
$dbCli = new db('clientes');
$clientes = $dbCli->all();
```

⚠ **Segurança:** A classe usa **Prepared Statements** internamente nos métodos store, update, find e destroy. NUNCA concatene variáveis diretamente no SQL — isso causa **SQL Injection**!

API REST com PHP Puro

Conceitos REST:

Método HTTP	Ação CRUD	Exemplo de URL
GET	Read (ler)	/api/servicos
POST	Create (criar)	/api/servicos
PUT	Update (atualizar)	/api/servicos/5
DELETE	Delete (deletar)	/api/servicos/5

api/servicos.php — usando db.class.php

```
<?php
header('Content-Type: application/json');
header('Access-Control-Allow-Origin: *');

require_once 'db.class.php';
$db      = new db('servicos');
$metodo  = $_SERVER['REQUEST_METHOD'];
$id      = $_GET['id'] ?? null;

switch ($metodo) {
    case 'GET':
        $data = $id ? $db->find($id) : $db->all();
        echo json_encode(['data' => $data]);
        break;
    case 'POST':
        $body = json_decode(file_get_contents('php://input'), true);
        $db->store($body);
        http_response_code(201);
        echo json_encode(['message' => 'Criado!']);
        break;
    case 'PUT':
        $body = json_decode(file_get_contents('php://input'), true);
        $db->update($id, $body);
        echo json_encode(['message' => 'Atualizado!']);
        break;
    case 'DELETE':
        $db->destroy($id);
        echo json_encode(['message' => 'Deletado!']);
        break;
}
```

Com a classe db, o CRUD completo fica limpo e reutilizável. Basta instanciar com o nome da tabela: new db('servicos'). O Slim vai organizar melhor ainda as rotas — próxima aula!

Organização de Código

include / require — reutilização de arquivos:

```
// include: aviso se não encontrar
include 'config.php';

// require: erro fatal se não encontrar (prefira!)
require 'db.class.php';

// _once: inclui apenas uma vez
require_once 'models/Servico.php';
```

Namespaces — organização modular:

```
// Em Models/Servico.php:
namespace App\Models;

class Servico { /* ... */ }

// Em outro arquivo:
use App\Models\Servico;
$s = new Servico("Teste", 10);
```

Autoload com Composer — sem require manual:

```
// composer.json
{
    "autoload": {
        "psr-4": {
            "App\\": "src/"
        }
    }
}
```

```
// index.php
require 'vendor/autoload.php';

// PHP encontra automaticamente qualquer classe
// com namespace App\ dentro de src/
use App\Controllers\ServicoController;
use App\Models\Servico;
```

O Composer é o gerenciador de dependências padrão do PHP — equivalente ao npm no JavaScript.

Tratamento de Erros e Exceções

excecoes.php

```
<?php
require_once 'db.class.php';

// try / catch / finally
try {
    $db      = new db('servicos');
    $result = $db->find(99);

    if (!$result) {
        throw new Exception("Serviço não encontrado", 404);
    }

    echo json_encode($result);

} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode([
        'erro' => 'Erro no banco de dados'
    ]);
} catch (Exception $e) {
    http_response_code($e->getCode() ?: 400);
    echo json_encode([
        'erro' => $e->getMessage()
    ]);
} finally {
    // Sempre executa (logs, limpeza)
    $db = null;
}
```

Códigos de status HTTP — use sempre!

Código	Significado
200 OK	Sucesso
201 Created	Criado com sucesso
400 Bad Request	Dados inválidos
404 Not Found	Recurso não existe
500 Server Error	Erro interno

```
// Definir código de status
http_response_code(201);

// Redirecionar
header('Location: /nova-pagina');
exit;
```

Boas Práticas e Segurança



SQL Injection

Sempre use Prepared Statements com PDO. Nunca concatene variáveis no SQL.



XSS

Use htmlspecialchars() ao exibir dados do usuário em HTML.



Validação

Use filter_var() para validar emails, URLs e inteiros. Nunca confie no cliente.



Estrutura

Separe configurações, modelos e controladores em arquivos diferentes.



Credenciais

Use variáveis de ambiente (.env) para senhas e chaves. Nunca no código.



PSR Standards

Siga as PSR-1, PSR-4 e PSR-12 para código legível e interoperável.

Regra de ouro: Nunca exiba mensagens de erro detalhadas em produção. Use logs no servidor.

Exercício Prático — Aula 1

Crie um arquivo PHP que simule uma mini-API de serviços **sem framework**:

1. Crie a tabela `servicos` no MySQL com os campos: `id`, `titulo`, `descricao`, `preco`, `criado_em`.
2. Crie `conexao.php` com a função `conectar()` usando PDO.
3. Crie `api/servicos.php` que responda ao método **GET** listando todos os serviços em JSON.
4. Adicione suporte ao método **POST** para cadastrar um novo serviço a partir de um JSON no body.
5. **Bônus:** suporte a `GET /api/servicos?id=1` para buscar um serviço específico.

Estrutura esperada da resposta:

```
{
  "success": true,
  "data": [
    { "id": 1, "titulo": "Hidráulica", "preco": "150.00" },
    { "id": 2, "titulo": "Elétrica", "preco": "200.00" }
  ]
}
```

Resumo — O que aprendemos



Sintaxe Básica

Tags PHP, echo, comentários, sensibilidade a maiúsculas.



Tipos e Variáveis

Tipagem dinâmica, casting, string, int, float, bool, array, null.



Controle de Fluxo

if/else, switch, for, foreach, while, match (PHP 8).



Funções

Declaração, parâmetros padrão, closures, arrow functions.



Banco de Dados

PDO, prepared statements, CRUD completo com segurança.



API REST

Headers HTTP, json_encode, métodos GET/POST/PUT/DELETE.

Próxima aula (16/06): Slim Framework — rotas, controllers, models e API REST profissional!

Referências e Recursos



PHP Manual Oficial

php.net/manual/pt_BR —
documentação completa em
português.



PHP The Right Way

phptherightway.com — guia de boas
práticas para PHP moderno.



Composer

getcomposer.org — gerenciador de
dependências PHP.



PDO

php.net/manual/pt_BR/book.pdo.php
— documentação da extensão PDO.

Repositório do projeto: http://localhost/pasta_projeto/ — veja os exemplos em `api/`